



Data Files Documentation

- [DataFile](#)
- [InvFile](#)

These classes are used to manage `.DAT` and `.INV` files for reading game data and reading and writing player inventories. In most cases, if a user just needs to add an item they don't need to use these `InvFile` methods directly — they can use the `player.additem()` helpers described in the [player documentation](#).

DataFile

Used to parse binary data files that hold game data.

data.ids

Property — Type: Uint32Array

Contains all of the form IDs in the data file.

new DataFile(path)

Constructor

Creates a new DataFile instance from a binary data file on disk.

Parameters

Name	Type	Default	Description
path	string	—	Path to the file on disk

Example

```
const data = new DataFile("/DATA/F3/MISC.DAT");
```

data.getId(formId)

Method — Returns: object

Looks up a record in the data file by its form ID and returns the item data as a JSON object.

Parameters

Name	Type	Default	Description
formId	number	—	The form ID to look up

data.close()

Safely closes the open file handle.

InvFile

The user's inventory for a given item type. Lists what items they have, the count of each, and the condition of each item.

```
new InvFile(path, options)
```

Creates a new InvFile instance from an inventory file on disk.

Parameters

Name	Type	Default	Description
<code>path</code>	<code>string</code>	—	Path to the inventory file on disk, e.g. <code>"/INV/F3/MISC.INV"</code>
<code>options</code>	<code>object</code>	—	An options object (see below)

Options

Property	Type	Default	Description
<code>onLoaded</code>	<code>function</code>	—	Callback function that is called once the file

Property	Type	Default	Description
			loads, receives an instance of <code>this</code> (the InvFile) as its parameter
<code>idOrder</code>	<code>Uint32Array</code>	—	An array of form IDs (typically from <code>data.ids</code>) that determines the sort order of the inventory — items will

Property	Type	Default	Description
			match the order of form IDs in this array

`inv.refreshItems()`

Method

Reads the file from disk. If an `onLoaded` callback was specified, it is called. This method is called automatically during the constructor.

`inv.ids()`

Method — Returns: `array`

Returns an array of all the form IDs present in this inventory file.

inv.get(i)

Method — Returns: object

Gets the inventory item at index position `i`. This is **not** the form ID — it's the index position in the inventory.

Parameters

Name	Type	Default	Description
<code>i</code>	number	—	The index position of the item

Returns

The returned object has the following fields:

Field	Type	Description
<code>id</code>	number	The form ID of the item

Field	Type	Description
<code>cnt</code>	<code>number</code>	The count of that item
<code>cnd</code>	<code>number</code>	The condition percentage (<code>0</code> – <code>100</code>)
<code>fl</code>	<code>number</code>	Flags — a <code>uint8</code> for binary flag use. Currently no flags are in use.

`inv.set(i, data)`

Method

Updates an existing inventory item at index position `i`. This does **not** add new items — it only updates existing ones.

Parameters

Name	Type	Default	Description
i	number	—	The index position of the item to update
data	object	—	An object with the fields to update (e.g. {cnt: 3} or {cnd: 50})

inv.add(data)

Method

Adds a new entry to the inventory. The data object must include at least id (the item's form ID). Can also include cnt, cnd, and fl.

Parameters

Name	Type	Default	Description
data	object	—	An object with fields for the new item

Defaults

Field	Default	Description
cnt	1	Number of items
cnd	100	Condition percentage (0 – 100)
fl	0	Flags (uint8)

inv.remove(i)

Method

Removes the inventory item at index position `i`.

Parameters

Name	Type	Default	Description
i	number	—	The index position of the item to remove

inv.indexOf(formId)

Method — Returns: number

Returns the index of where a given form ID appears in the inventory.

Parameters

Name	Type	Default	Description
formId	number	—	The form ID to find

inv.sync()

Method

Flushes any modifications to disk. Won't write if there were no changes. Should only be called when the page exits via `remove()`, or as infrequently as possible to reduce disk thrashing.